

INTRODUCTION TO MACHINE LEARNING

Math and Python: Your Computational Toolkit

Vectors, matrices, gradients and pandas, with
every idea written twice: once in notation, once
in code.

Every Idea, Written Twice

NOTATION

$$\theta^T x$$

Compact, exact, and the way papers and textbooks will speak to you.

CODE

```
theta @ x
```

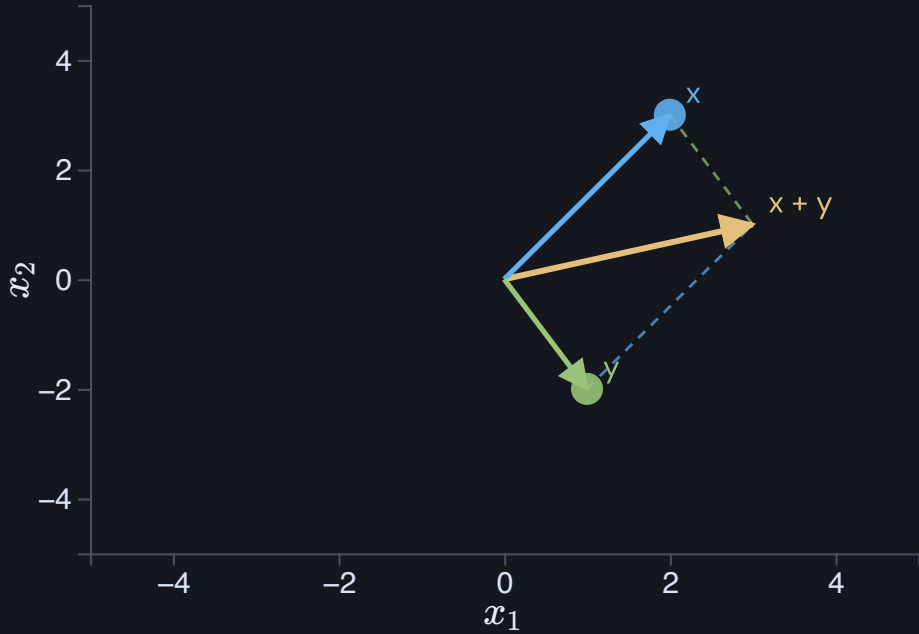
Runnable, checkable, and the way you will actually test an idea.

This session is a **translation exercise**. Every symbol you meet has a line of code, and every line of code has a symbol.

BRING YOUR LAPTOP

The five notebooks are the lab. These slides are the map.

Vectors



drag either arrow head

DEFINITION

An ordered list of numbers, $x \in \mathbb{R}^n$.

$$x = [x_1, x_2, \dots, x_n]^\top$$

GEOMETRY

A direction with a length. Addition is placing them head to tail; the sum is the diagonal.

IN MACHINE LEARNING

One example is a vector. One row of your data table.

Vector Operations

```
1 import numpy as np
2
3 x = np.array([2, 3, 1])
4 y = np.array([1, -2, 4])
5 print(x.shape, len(x))
6
7 # addition and scaling
8 print(x + y)
9 print(3 * x)
10
11 # the inner product
12 print(np.dot(x, y))
13 print(np.linalg.norm(x))
```

`x.shape, len(x)`

`(3,) 3`

`# a 3-vector: three numbers`
`# in a fixed order`

The inner product $x^\top y = \sum_i x_i y_i$ is the single most common operation in this course.

Vector Operations

```
1 import numpy as np
2
3 x = np.array([2, 3, 1])
4 y = np.array([1, -2, 4])
5 print(x.shape, len(x))
6
7 # addition and scaling
8 print(x + y)
9 print(3 * x)
10
11 # the inner product
12 print(np.dot(x, y))
13 print(np.linalg.norm(x))
```

`x.shape, len(x)`

`(3,) 3`

`# a 3-vector: three numbers`
`# in a fixed order`

The inner product $x^\top y = \sum_i x_i y_i$ is the single most common operation in this course.

Vector Operations

```
1 import numpy as np
2
3 x = np.array([2, 3, 1])
4 y = np.array([1, -2, 4])
5 print(x.shape, len(x))
6
7 # addition and scaling
8 print(x + y)
9 print(3 * x)
10
11 # the inner product
12 print(np.dot(x, y))
13 print(np.linalg.norm(x))
```

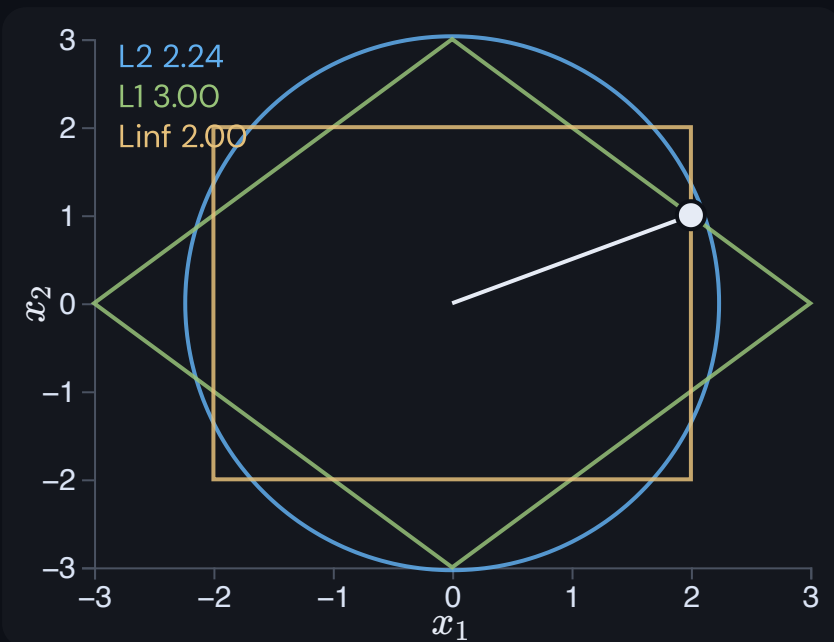
`x.shape, len(x)`

`(3,) 3`

`# a 3-vector: three numbers`
`# in a fixed order`

The inner product $x^\top y = \sum_i x_i y_i$ is the single most common operation in this course.

How Long, How Far, How Similar



drag the point: one vector, three lengths

THREE NORMS

$$\|x\|_2 = \sqrt{\sum_i x_i^2} \quad \|x\|_1 = \sum_i |x_i| \quad \|x\|_\infty = \max_i |x_i|$$

SQUARED LENGTH IS A DOT PRODUCT

$\|x\|_2^2 = x^\top x$, and the distance between two examples is $\|x - y\|_2$. Every squared-error loss is that one quantity.

ANGLE, NOT JUST LENGTH

$$\cos \theta = \frac{x^\top y}{\|x\|_2 \|y\|_2}$$

A zero dot product means perpendicular. This is how embedding models compare two things.

WHICH ONE TO MINIMISE

$\|\cdot\|_2$ spreads error evenly; $\|\cdot\|_1$ tolerates a few large mistakes and pushes the rest to zero.

Matrices

DEFINITION

A rectangular array, $A \in \mathbb{R}^{m \times n}$: m rows, n columns.

IN MACHINE LEARNING

Your whole dataset: one row per example, one column per feature.

$$X \in \mathbb{R}^{m \times d}$$

ALSO

A linear transformation: Ax sends a vector to another vector.



$C = AB$: **hover a cell of C** to see which row and column produce it

Matrices in NumPy

```
1 A = np.array([[1, 2, 3],
2               [4, 5, 6]])
3 print(A.shape)    # (2, 3)
4 print(A.size)     # 6
5
6 B = np.zeros((3, 3))
7 C = np.ones((2, 4))
8 D = np.eye(3)
9 E = np.random.rand(2, 3)
10
11 # the two products people confuse
12 print(A * A)      # elementwise
13 print(A @ A.T)    # matrix product
```

```
A.shape (2, 3)
```

```
A.size 6
```

```
[[1 2 3]
 [4 5 6]]
```

THE CLASSIC BUG

A * B multiplies elementwise; **A @ B** is the matrix product. They are different operations, and NumPy will happily run the wrong one.

Matrices in NumPy

```
1 A = np.array([[1, 2, 3],
2               [4, 5, 6]])
3 print(A.shape)    # (2, 3)
4 print(A.size)     # 6
5
6 B = np.zeros((3, 3))
7 C = np.ones((2, 4))
8 D = np.eye(3)
9 E = np.random.rand(2, 3)
10
11 # the two products people confuse
12 print(A * A)      # elementwise
13 print(A @ A.T)    # matrix product
```

```
A.shape (2, 3)
```

```
A.size 6
```

```
[[1 2 3]
 [4 5 6]]
```

THE CLASSIC BUG

A * B multiplies elementwise; **A @ B** is the matrix product. They are different operations, and NumPy will happily run the wrong one.

Matrices in NumPy

```
1 A = np.array([[1, 2, 3],
2               [4, 5, 6]])
3 print(A.shape)    # (2, 3)
4 print(A.size)     # 6
5
6 B = np.zeros((3, 3))
7 C = np.ones((2, 4))
8 D = np.eye(3)
9 E = np.random.rand(2, 3)
10
11 # the two products people confuse
12 print(A * A)      # elementwise
13 print(A @ A.T)    # matrix product
```

A.shape (2, 3)

A.size 6

```
[[1 2 3]
 [4 5 6]]
```

THE CLASSIC BUG

A * B multiplies elementwise; **A @ B** is the matrix product. They are different operations, and NumPy will happily run the wrong one.

Two Ways to Read Ax

ROW PICTURE

Each entry of the output is one dot product:

$$(Ax)_i = a_i^\top x$$

Row i of A asks one question about x .

COLUMN PICTURE

The output is a weighted sum of the columns:

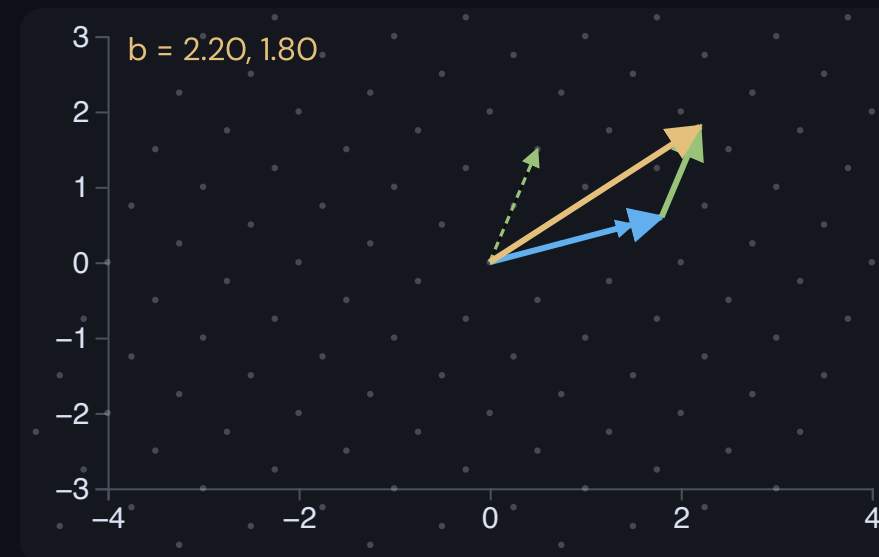
$$Ax = x_1 a_{(1)} + x_2 a_{(2)} + \cdots + x_n a_{(n)}$$

x is a recipe; the columns are the ingredients.

THE MATRIX IN THE PLOT

$$A = \begin{bmatrix} 1.5 & 0.5 \\ 0.5 & 1.5 \end{bmatrix}, \quad x = \begin{bmatrix} x_1 \\ x_2 \end{bmatrix}$$

The **first column** and the **second** are the two dashed arrows.



dashed: columns of A · solid: scaled by the sliders · yellow: their sum

x_1 1.2 x_2 0.8

make the columns dependent

WHY THE SECOND ONE MATTERS

Every vector you can reach is a combination of the columns. That set is the **column space**, and $Ax = b$ is solvable exactly when b lies in it.

Span, Independence, Rank

- **Span.** Everything you can build from a set of vectors by scaling and adding. Two independent vectors in the plane span the plane; two parallel ones span only a line.
- **Linear independence.** No vector in the set is a combination of the others, which is the same as saying only the zero recipe reaches zero.

$$Xv = 0 \implies v = 0$$

- **Rank.** The number of independent columns, so the dimension of the column space. It can never exceed $\min(m, n)$.

THE SENTENCE TO REMEMBER

$$\text{rank}(X) = n \iff \text{columns independent} \iff X^\top X \text{ invertible}$$

Inverse, Determinant, and When Things Break

- **Inverse.** The matrix that, when multiplied by A , is the identity (i.e. 1). It solves a linear system.

$$AA^{-1} = A^{-1}A = I, \quad Ax = b \Rightarrow x = A^{-1}b$$

- **Determinant.** The factor by which A scales volume. Zero means the map flattens space, and nothing can unflatten it.

$$\det(A) = 0 \iff A \text{ has no inverse}$$

- **When an inverse exists.** Square, and full rank: no column is a combination of the others. x
- **Why you will care.** The normal equations need this inverse. Duplicate a feature and it stops existing.

$$\theta = (X^{\top}X)^{-1}X^{\top}y$$

IN PRACTICE

Use `np.linalg.solve(A, b)`, instead of `np.linalg.inv(A) @ b`. Never invert a matrix! (as a rule of thumb)

When the Inverse Quietly Lies

Real measurements, 342 penguins, four features plus an intercept column. Now add one more column: bill length again, in centimetres.

```
1 X = np.column_stack([np.ones(342), bill_len, bill_dep,
2                       flipper, mass])
3
4 np.linalg.matrix_rank(X)      # 5 of 5 columns
5 np.linalg.cond(X.T @ X)      # 1.8e+10
6
7 # the same feature, in different units
8 X2 = np.column_stack([X, bill_len / 10])
9
10 np.linalg.matrix_rank(X2)     # 5 of 6 columns
11 np.linalg.cond(X2.T @ X2)     # 7.1e+21
12
13 # and this does not raise
14 theta = np.linalg.inv(X2.T @ X2) @ X2.T @ y
15 np.linalg.det(X2.T @ X2)      # -7.9e+09, not 0
```

```
matrix_rank(X)
```

```
5
```

```
cond(X.T @ X)
```

```
1.8e+10
```

```
# five independent columns:
```

```
# the fit is well posed
```

THE LESSON

Singular matrices rarely announce themselves. Check `matrix_rank` and `cond`, and prefer `lstsq`.

When the Inverse Quietly Lies

Real measurements, 342 penguins, four features plus an intercept column. Now add one more column: bill length again, in centimetres.

```
1 X = np.column_stack([np.ones(342), bill_len, bill_dep,
2                       flipper, mass])
3
4 np.linalg.matrix_rank(X)      # 5 of 5 columns
5 np.linalg.cond(X.T @ X)      # 1.8e+10
6
7 # the same feature, in different units
8 X2 = np.column_stack([X, bill_len / 10])
9
10 np.linalg.matrix_rank(X2)     # 5 of 6 columns
11 np.linalg.cond(X2.T @ X2)     # 7.1e+21
12
13 # and this does not raise
14 theta = np.linalg.inv(X2.T @ X2) @ X2.T @ y
15 np.linalg.det(X2.T @ X2)      # -7.9e+09, not 0
```

```
matrix_rank(X)
```

```
5
```

```
cond(X.T @ X)
```

```
1.8e+10
```

```
# five independent columns:
```

```
# the fit is well posed
```

THE LESSON

Singular matrices rarely announce themselves. Check `matrix_rank` and `cond`, and prefer `lstsq`.

When the Inverse Quietly Lies

Real measurements, 342 penguins, four features plus an intercept column. Now add one more column: bill length again, in centimetres.

```
1 X = np.column_stack([np.ones(342), bill_len, bill_dep,
2                       flipper, mass])
3
4 np.linalg.matrix_rank(X)      # 5 of 5 columns
5 np.linalg.cond(X.T @ X)      # 1.8e+10
6
7 # the same feature, in different units
8 X2 = np.column_stack([X, bill_len / 10])
9
10 np.linalg.matrix_rank(X2)    # 5 of 6 columns
11 np.linalg.cond(X2.T @ X2)    # 7.1e+21
12
13 # and this does not raise
14 theta = np.linalg.inv(X2.T @ X2) @ X2.T @ y
15 np.linalg.det(X2.T @ X2)     # -7.9e+09, not 0
```

```
matrix_rank(X)
```

```
5
```

```
cond(X.T @ X)
```

```
1.8e+10
```

```
# five independent columns:
```

```
# the fit is well posed
```

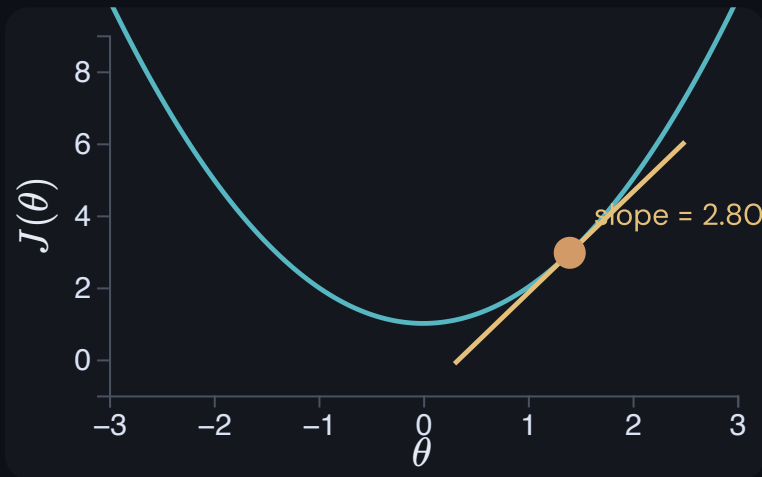
THE LESSON

Singular matrices rarely announce themselves. Check `matrix_rank` and `cond`, and prefer `lstsq`.

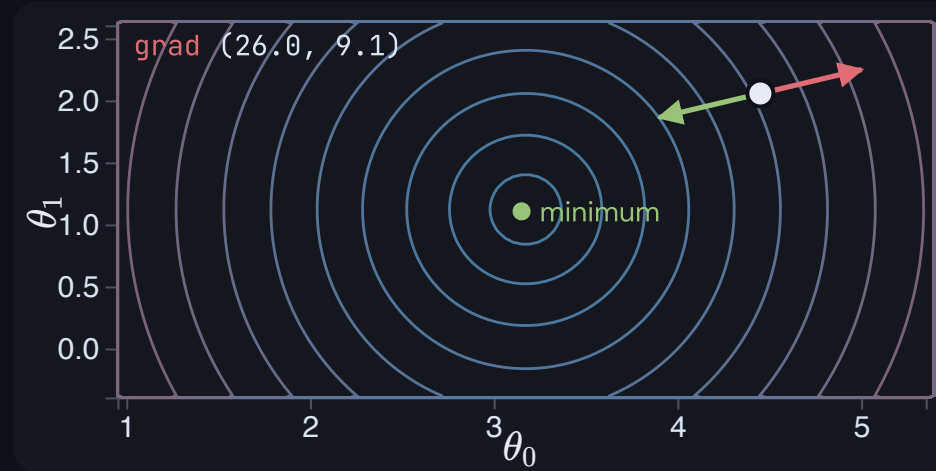
The Calculus You Actually Need

- **Derivative.** For $J : \mathbb{R} \rightarrow \mathbb{R}$, the slope: how much the output moves when the input nudges.
- **Gradient.** For $J : \mathbb{R}^d \rightarrow \mathbb{R}$, one partial derivative per parameter, collected into a vector that points *uphill*.

$$\nabla_{\theta} J = \left[\frac{\partial J}{\partial \theta_1}, \dots, \frac{\partial J}{\partial \theta_d} \right]^{\top} \in \mathbb{R}^d$$



drag to see the slope



Contours of the parabolic least-squares cost. **drag** to see the steepest ∇J (ascent) and $-\nabla J$ (descent)

COMMON EXPRESSIONS

$\nabla_{\theta}(\theta^{\top} x) = x \nabla_{\theta} \|\theta\|^2 = 2\theta$, and the chain rule.

The One Gradient You Will Use All Semester

Least squares in matrix form, differentiated once, slowly. Here $X \in \mathbb{R}^{m \times d}$, $y \in \mathbb{R}^m$, $\theta \in \mathbb{R}^d$, and $J(\theta)$ is a single number.

- 1 Expand the squared norm.

$$J(\theta) = \|X\theta - y\|_2^2 = \theta^\top X^\top X\theta - 2y^\top X\theta + y^\top y$$

- 2 Apply two rules. $A = X^\top X$ is symmetric by construction.

$$\nabla_\theta(b^\top \theta) = b, \quad \nabla_\theta(\theta^\top A\theta) = 2A\theta$$

- 3 Assemble, and factor.

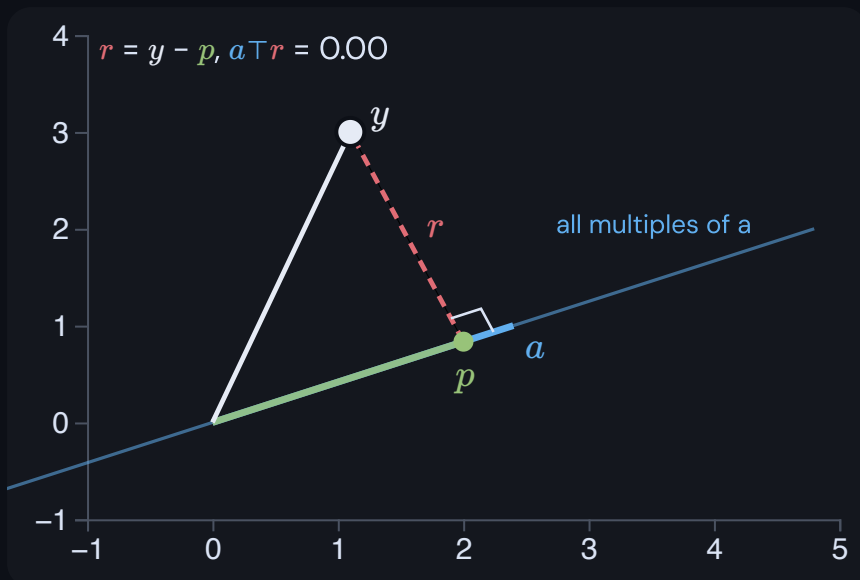
$$\nabla_\theta J = 2X^\top X\theta - 2X^\top y = 2X^\top (X\theta - y) \in \mathbb{R}^d$$

- 4 Set it to zero. These are the normal equations, and tomorrow's lecture is mostly about this line.

$$X^\top X\theta = X^\top y$$

Read it as a sentence: $X\theta - y$ is the residual, and $X^\top$ asks each feature how much of that residual it can still explain.

Least Squares Is a Projection



drag y . The plane of the page stands for $\mathbb{R}^m$, one axis per data point.

- **Onto one direction.** With $a, y \in \mathbb{R}^m$, the part of y that a can explain is

$$p = \frac{a^\top y}{a^\top a} a$$

- **Onto every direction at once.** The columns of $X \in \mathbb{R}^{m \times d}$ span a d -dimensional slab inside $\mathbb{R}^m$. Every prediction $X\theta$ lives in it, and no choice of θ escapes. The closest reachable point to y is its shadow on that slab.

$$\hat{y} = X \underbrace{(X^\top X)^{-1} X^\top}_{\theta} y$$

- **Why the residual is perpendicular.** If it leaned at all, some column could still explain part of it and you could do better. So

$$X^\top (y - X\theta) = 0 \iff X^\top X\theta = X^\top y$$

Broadcasting, and the Bug It Causes

THE RULE

Line the shapes up from the right. Two dimensions are compatible when they are equal, or one of them is 1, and a 1 is stretched to match.

a.shape	b.shape	a + b
(342, 4)	(4,)	(342, 4)
(342, 4)	(342, 1)	(342, 4)
(342, 4)	(342,)	error
(342,)	(342, 1)	(342, 342)

STANDARDIZING, CORRECTLY

```
mu = X.mean(axis=0)      # (4,)  
sd = X.std(axis=0)       # (4,)  
Z  = (X - mu) / sd       # (342, 4)
```

THE LAST ROW OF THAT TABLE

Predictions of shape $(m,)$ minus targets of shape $(m, 1)$ give you an $m \times m$ matrix, no error, and a loss that is quietly meaningless. **Print shapes when a number surprises you.**

Avoid Loops if You Can! Vectorize.

ONE MILLION-ELEMENT DOT PRODUCT

Python loop	138.5 ms
<code>a @ b</code>	0.76 ms
speedup	182 times

Measured on an Apple M-series laptop, NumPy 1.26, same answer to nine decimal places. A 500 by 500 matrix product takes 15.5 ms, about 16 GFLOP/s.

WHERE THE TIME GOES

The loop spends it on Python objects, one boxed float at a time. NumPy hands the whole array to compiled code that uses your CPU's vector instructions.

THE HABIT

If you are writing `for i in range(len(x))` over data, there is almost always an array expression that replaces it, and it is both faster and closer to the formula on the slide.

Reading and Exploring Data

```
1 import pandas as pd
2
3 df = pd.read_csv("penguins.csv")
4 print(df.shape)
5
6 print(df.head(3))
7 print(df.describe())
8 print(df.isnull().sum())
9
10 # the four numeric columns, as one matrix
11 cols = ["bill_length_mm", "bill_depth_mm",
12         "flipper_length_mm", "body_mass_g"]
13 X = df[cols].dropna().values
14 print(X.shape, X.dtype)
```

```
df.shape
(344, 7)
```

```
# 344 penguins, 7 columns:
# 3 labels, 4 measurements
```

Always `describe()` and check for missing values before modelling anything. Two of these 344 penguins were never measured, and `dropna()` is a decision, not a formality.

Reading and Exploring Data

```
1 import pandas as pd
2
3 df = pd.read_csv("penguins.csv")
4 print(df.shape)
5
6 print(df.head(3))
7 print(df.describe())
8 print(df.isnull().sum())
9
10 # the four numeric columns, as one matrix
11 cols = ["bill_length_mm", "bill_depth_mm",
12         "flipper_length_mm", "body_mass_g"]
13 X = df[cols].dropna().values
14 print(X.shape, X.dtype)
```

```
df.shape
(344, 7)
```

```
# 344 penguins, 7 columns:
# 3 labels, 4 measurements
```

Always `describe()` and check for missing values before modelling anything. Two of these 344 penguins were never measured, and `dropna()` is a decision, not a formality.

Reading and Exploring Data

```
1 import pandas as pd
2
3 df = pd.read_csv("penguins.csv")
4 print(df.shape)
5
6 print(df.head(3))
7 print(df.describe())
8 print(df.isnull().sum())
9
10 # the four numeric columns, as one matrix
11 cols = ["bill_length_mm", "bill_depth_mm",
12         "flipper_length_mm", "body_mass_g"]
13 X = df[cols].dropna().values
14 print(X.shape, X.dtype)
```

```
df.shape
(344, 7)
```

```
# 344 penguins, 7 columns:
# 3 labels, 4 measurements
```

Always `describe()` and check for missing values before modelling anything. Two of these 344 penguins were never measured, and `dropna()` is a decision, not a formality.

The Design Matrix

Every dataset in this course arrives in this one shape: m examples down, d features across, and the thing you want to predict pulled out into its own column.

	bill length	bill depth	flipper	body mass	
1 Adelie	39.1	18.7	181.0	3750	
2 Adelie	39.5	17.4	186.0	3800	
3 Adelie	40.3	18.0	195.0	3250	← one penguin
4 Adelie	36.7	19.3	193.0	3450	
5 Adelie	39.3	20.6	190.0	3650	
⋮	⋮	⋮	⋮	⋮	
342 Gentoo	49.9	16.1	213.0	5400	

X 342 rows × 3 columns

y 342 rows

one column: one measurement, all 342 penguins

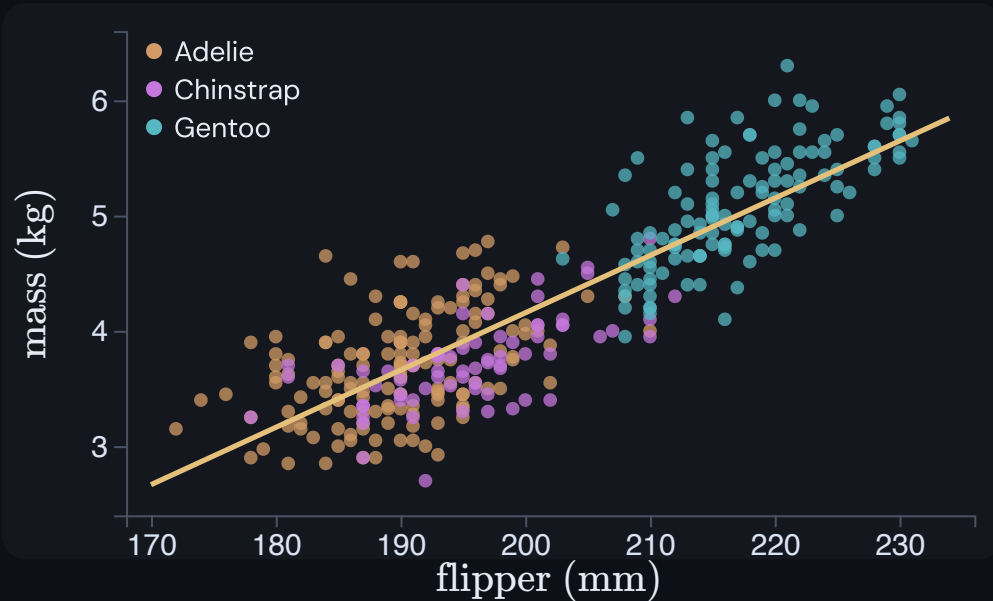
A ROW

One penguin. $x_i^\top \in \mathbb{R}^{1 \times d}$, a point in feature space.

A COLUMN

One measurement across every penguin. $X_{:,j} \in \mathbb{R}^m$, the vector a single weight θ_j multiplies.

Statistics Is Linear Algebra



Least squares fit, in grams

$$\text{mass} \approx -5781 + 49.7 \times \text{flipper}$$

$$R^2 = 0.76$$

FIRST, PUT THE COLUMNS ON ONE SCALE

Subtract each column's mean and divide by its standard deviation, giving $Z \in \mathbb{R}^{342 \times 4}$. Millimetres and grams are now comparable.

THEN ONE PRODUCT DOES THE REST

$$C = \frac{1}{m-1} Z^T Z \in \mathbb{R}^{4 \times 4}$$

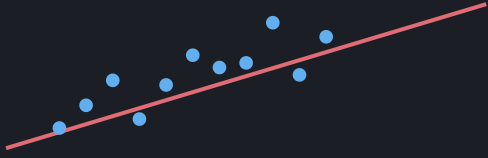
Every pairwise correlation, from one matrix multiplication. Entry C_{jk} is column j dotted with column k .

	bill L	bill D	flipper	mass
bill L	1.00	-0.24	0.66	0.60
bill D	-0.24	1.00	-0.58	-0.47
flipper	0.66	-0.58	1.00	0.87
mass	0.60	-0.47	0.87	1.00

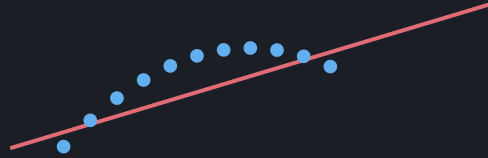
Palmer Station penguins (Gorman et al.), CCO, vendored with the deck.

Look at It Before You Model It

set 1



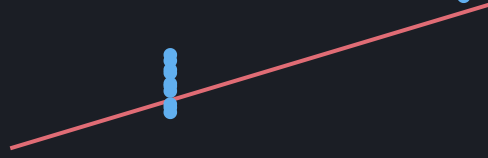
set 2



set 3



set 4



SAME STATISTICS

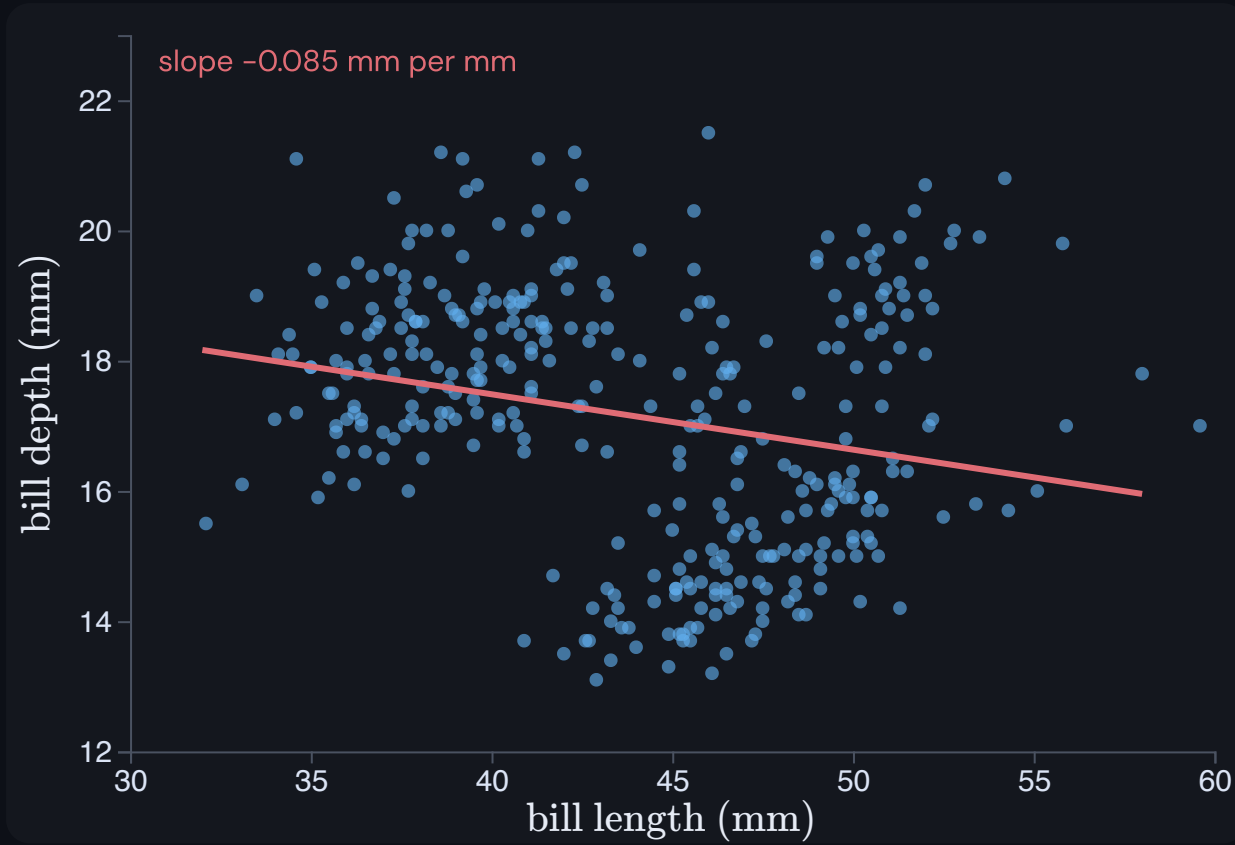
These four datasets share their means, variances, and correlation to two decimal places.

DIFFERENT WORLDS

A line fitted to all four is identical, and wrong for three of them. **Plot your data.**

Anscombe's quartet, 1973

Two Measurements, 342 Birds

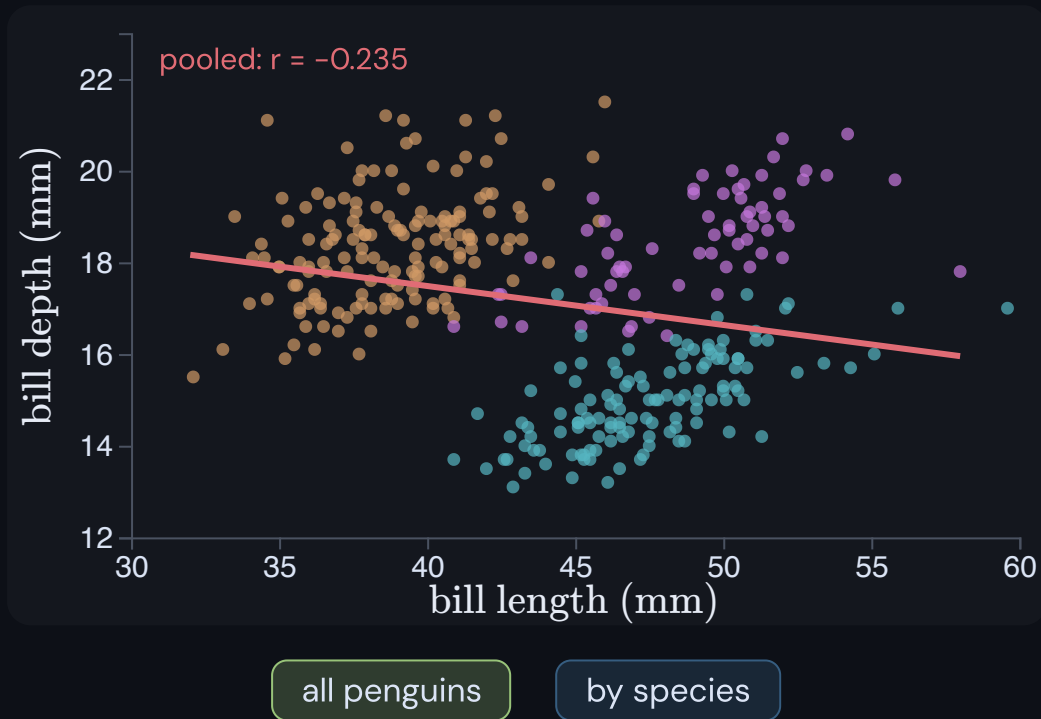


- **The number.** Bill length against bill depth, every bird in the file, and the line least squares puts through them.

$$r = -0.235$$

- **The claim it invites.** Longer bills are shallower.
- **Before you believe it.** What would you want to see that this plot is not showing you?

The Same Data, Two Answers



POOLED

Bill length against bill depth over all 342 birds:
 $r = -0.235$. Longer bills look shallower.

WITHIN EACH SPECIES

Adelie $r = +0.39$, Chinstrap $+0.65$, Gentoo $+0.64$. Every group says the opposite of the pool.

SIMPSON'S PARADOX

A missing variable, species here, can reverse the sign of a relationship. No amount of linear algebra warns you. Only the plot does.

The Lab: Five Notebooks

- **00 Python intro**: syntax, collections, functions
- **01 Vectors**: creation, arithmetic, inner products, norms
- **02 Matrices**: indexing, slicing, the two products
- **03 Matrix properties and calculus**: inverse, determinant, rank, gradients
- **04 Data reading and plotting**: pandas, summary statistics, figures

HOW TO USE THEM

Run every cell. Change a number and predict what happens before you press shift-enter. That prediction is the learning.

Summary

VECTORS

$$x^\top y = \sum_i x_i y_i$$

one example, one row

MATRICES

$$X \in \mathbb{R}^{m \times d}$$

the dataset, and a transformation

GRADIENTS

$$\nabla_\theta J$$

the direction learning walks against

AND ALWAYS

look at the data before you model it

Next: supervised learning and linear regression, where all of this becomes one line of code.